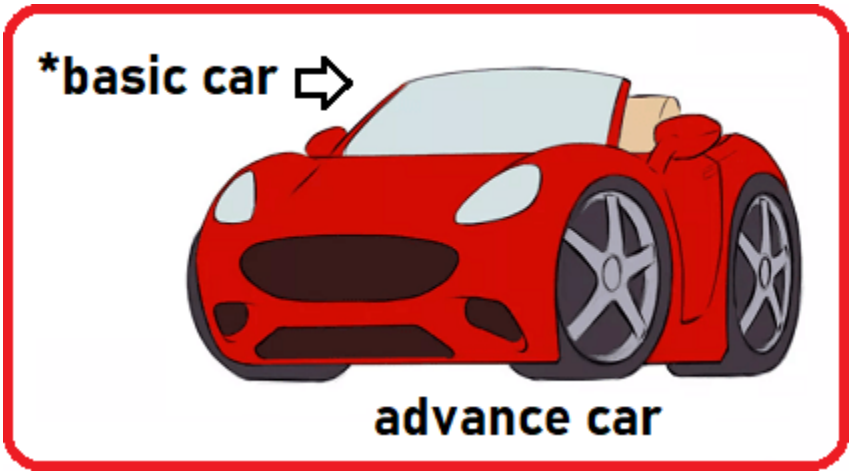
## Real-Time Example to Understand the need for Virtual Function :




Pointer is the basic car but the actual object is the advanced car. So, I am thinking of it as a basic car. How can I drive? How it will run? Is it will run like what I am thinking or run like an advanced car? It will run like an advanced car because I am pointing at the object of an advanced car.

So, when you call a function that is present in the base case as well as the derived class, whose function must be called? Based on the object the function must be called. Not based on the pointer.

See it is just like if you saw a donkey and say it's a horse, will it run like a horse? You are thinking of it like a horse but that's a donkey. It cannot run like a horse. In the same way, here the basic car is the reference and you are pointing at the advanced car object. So, the object is of the derived class and the pointer is of the base class.

When we call a function then the base class function is called. But logically it is wrong. Whose function must be called? The function must be called based on the object.

## Key points about virtual functions :-

1. Virtual functions are used for achieving polymorphism
2. The base class can have virtual functions
3. Virtual functions **can be** overrides in the derived class
4. Pure virtual functions **must be** overrides by the derived class

## Rules for virtual functions :-

1. Virtual functions in C++ cannot be static.
2. A virtual function in C++ can also be a friend function of another class.
3. The prototype/Signature of virtual functions should be the same in the base as well as the derived class.
4. In C++, a class may have a virtual destructor but it cannot have a virtual constructor.

## Limitation of virtual functions :-

- Slower
- Difficult to debug

## Runtime Polymorphism :-

```cpp
#include<bits/stdc++.h>
using namespace std;
class Car{
public:
    void Start(){cout<<"Car Started"<<endl;}
    void Stop(){cout<<"Car Stopped"<<endl;}
};
class Innova:public Car{
public:
    void Start(){cout<<"Innova Started"<<endl;}
    void Stop(){cout<<"Innova Stopped"<<endl;}
};

class Swift:public Car{
public:
    void Start(){cout<<"Swift Started"<<endl;}
    void Stop(){cout<<"Swift Stopped"<<endl;}
};
int main(){
    Car *c = new Innova();
    c->Start();
    c->Stop();
    c = new Swift();
    c->Start();
    c->Stop();
    return 0;
}
//Output
Car Started
Car Stopped
Car Started
Car Stopped
```

Now, suppose we want the derived class function should be called then we need to make the base class functions as virtual functions. This means if we want the Innova class Start function should be called then we have to make the Car class Start function virtual.

```cpp
#include<bits/stdc++.h>
using namespace std;
class Car{
public:
    virtual void Start(){cout<<"Car Started"<<endl;}
    virtual void Stop(){cout<<"Car Stopped"<<endl;}
};
class Innova:public Car{
public:
    void Start(){cout<<"Innova Started"<<endl;}
    void Stop(){cout<<"Innova Stopped"<<endl;}
};

class Swift:public Car{
public:
    void Start(){cout<<"Swift Started"<<endl;}
    void Stop(){cout<<"Swift Stopped"<<endl;}
};
int main(){
    Car *c = new Innova();
    c->Start();
    c->Stop();
    c = new Swift();
    c->Start();
    c->Stop();
    return 0;
}
//Output
Innova Started
Innova Stopped
Swift Started
Swift Stopped
```

## Key Points of Runtime Polymorphism in C++:

- Same name different actions
- Runtime Polymorphism is achieved using function overriding
- Virtual functions are abstract functions of the base class
- The derived class must override the virtual functions
- A base class pointer pointing to a derived class object and an override function is called

## Abstract class in C++

An abstract class in C++ has at least one pure virtual function by definition. In other words, a function that has no definition and these classes cannot be instantiated. The abstract class's child classes must provide body to the pure virtual function; otherwise, the child class would become an abstract class in its own right.

## What is the purpose of the Pure Virtual function?

The purpose of a pure virtual function is to achieve polymorphism. We want the derived classes to override this function. So, it becomes mandatory for derived classes to override the pure virtual function. It means the base class is governing or giving a command to the child classes that you must override the pure virtual functions. So, it is defining an interface.

If a class is having a pure virtual function, then that class is called an **Abstract Class in C++.**

**Can we create the object of Abstract Class in C++? No.**

**Can we create the pointer of the abstract class? Yes.**


## What is the purpose of inheritance?

- Two things, one is reusability. The derived class gets the function from the base class.
- The second thing to achieve is polymorphism.

| Base (All Concrete) | Base (Some Concrete) (Some Pure Virtual Func) | Base (All Pure Virtual Func) |
|---|---|---|
| \| | \| | \| |
| Reusability | Reusability + Polymorphism | Polymorphism |

## Key Points of Abstract class in C++:

- The class having a pure virtual function is an abstract class.
- An **Abstract class** can have concrete also.
- The object of abstract class cannot be created
- A derived class must override pure virtual function, otherwise, it will also become an abstract class.
- The pointer of the abstract class can be created.
- The pointer of the abstract class can hold the object of the derived class.
- Abstract classes are used for achieving polymorphism
- Abstract with some concrete and some pure virtual functions.

```cpp
#include<bits/stdc++.h>
using namespace std;
class Shape{
public:
    virtual float Area() = 0;
    virtual float Perimeter() = 0;
};
class Rectangle:public Shape{
private:
    float length;
    float breadth;
public:
    Rectangle(int l = 1, int b = 1){
        length = 1;
        breadth = b;
    }
    float Area(){return length * breadth;}
    float Perimeter(){return 2 * (length + breadth);}
};

class Circle:public Shape{
private:
    float radius;
public:
    Circle(float r){radius = r;}
    float Area(){return 3.1425 * radius * radius;}
    float Perimeter(){return 2 * 3.1425 * radius;}
};

int main(){
    Shape *s = new Rectangle(10, 5);
    cout<<"Area of Rectangle "<<s->Area()<<endl;
    cout<<"Perimeter of Rectangle "<<s->Perimeter()<<endl;
    s = new Circle(10);
    cout<<"Area of Circle "<<s->Area()<<endl;
    cout<<"Perimeter of Circle "<<s->Perimeter()<<endl;
}


```